Snapchat Political Ads

By: Jenna Yang and Alicia Gunawan

Introduction

`PoliticalAds_2018` and `PoliticalAds_2019` are datasets that provide details regarding political advertisements on Snapchat. These ads range from advertising for general elections, referendums and ballot measures, to ads about social issues. The dataset (2018 and 2019 combined) includes information about each ad, including who paid for the ad, under which organization, how much was spent, and how many views, or “impressions”, the ad received, among other details.

The question we are investigating is the following: Do the most popular ads cost more than the less popular ones? To help us determine this, we have chosen to focus mainly on examining the “Spend” and “Impressions” columns in the dataset. We wish to explore potential correlations between money spent and the relative success of the ad through impressions.

Cleaning and EDA

Our question centers around the popularity of the ad (measured in impressions) and the cost of the ad, so we also chose to explore the organization and paying advertiser of each ad to get a more clear sense of the people/functions behind the ad in relation to funding it and running it.

We started by first trying to deal with NaN values, through which we discovered that thankfully, the dataset did not contain any NaN values for our relevant columns.

We found that the ads were paid for in different currencies, and so it was necessary to convert the Spend column to one currency in order to accurately analyze and compare quantities. Thus, using the relevant conversion rates and averaging them for 2018 and 2019, we were able to convert the Spend column into all US dollars.

Univariate analysis: We specifically looked into the distributions of each of our focus columns, Spend and Impressions. We found that the median number of impressions was roughly 72000 impressions, so we decided to split and categorize the data into two categories: popular ads (ads with 72000+ impressions) and unpopular ads (ads with less than 72000 impressions). This categorization will provide a basis for our hypothesis testing.

Bivariate Analysis: Next, we plotted Impressions vs. Spend on a scatter plot to visually infer any potential associations. While there generally seems to be a positive correlation on the scatterplot, the strength of these seems to decrease as the number of impressions decreases. Regardless, such an association needs to be further tested with hypothesis testing. We also looked at Impressions and Spend in relation to organization and paying advertiser respectively. With these plots, we wished to answer questions that include:

- Which organizations got the most impressions in total?
- What about per ad?
- Which advertisers paid the most for ads?
- Which ones on average paid the most per ad?

In answering these questions, we found that a standout organization/advertiser was General Mills, who fund their own ads and by far had the most impressions and spent the most money both in total and per ad.

Aggregation: Lastly, we made pivot tables putting our variables together, and examined the averages and totals. It was interesting to see that Mike Bloomberg was one of the advertisers who spent the most per ad, but overall, did not show up in the top totals. When plotting the top 10 advertisers per ad, again, we see a generally positive correlation between impressions and spend. General Mills and Mike Bloomberg are far away from the rest, with both a large number of impressions and a large $ spent, suggesting a potential correlation which we plan to test in our hypothesis testing.

Assessment of Missingness

We narrowed down the columns with missingness that was not explicitly explained by the data dictionary. Of these columns, we can still identify columns that are missing by design—for example, missing values in `EndDate` would be because the ad is either 'run indefinitely or until an Advertiser pauses the campaign' according to Snapchat's guide (link at the bottom).

We chose the columns `Segments` and `CandidateBallotInformation`, both of which we tested against `Spend` and `Impressions`. We set the significance level to 0.1.

Segments

We chose to test `Segments` against `Spend` and `Impressions`, since we decided those columns were most relevant—for example, we thought that the greater reach an ad has or the more an organization spends, the more likely it will not leave `Segments` out.

The permutation test had `N=200` repetitions, and the p-values turned out to be:

- `Spend` : 0.4
- `Impressions` : 0.58

Neither p-value is less than our significance level (0.1), so we concluded that the correlation between `Segments` and `Spend` / `Impressions` is **not statistically significant**. This would mean that the missingness mechanism for `Segments` is NMAR, since it is not dependent on any column in the dataset.

While we can only speculate, perhaps further data on marketing strategies would help us explain the missingness in this column. For example, knowing if an organization has historically launched ads with very broad demographics (or even just the general audience) and thus no targeting criteria might be the reason why a value in `Segments` is missing.

CandidateBallotInformation

Again, we conducted a permutation test for `CandidateBallotInformation` against `Spend` and `Impressions`, with `N=200` repetitions. This time, the p-values were as follows:

- `Spend` : 0.305
- `Impressions` : 0.075

While the p-value of `Spend` is above the significance level, the p-value for comparing `Impressions` is 0.075, which less than 0.1. This means that the correlation between `Spend` and `CandidateBallotInformation` is **not statistically significant**, while the correlation between `Impressions` and `CandidateBallotInformation` **is statistically significant**. As such,

we concluded that the missingness mechanism for `CandidateBallotInformation` is MAR, and that it is dependent on the column `Impressions`.

Snapchat Ad Guide: [https://businesshelp.snapchat.com/en-US/article/political-ads-library (https://businesshelp.snapchat.com/en-US/article/political-ads-library)](https://businesshelp.snapchat.com/en-US/article/political-ads-library)

Hypothesis Test

We have decided to investigate whether or not an ad with more impressions has had more spending associated with it. Since the spread of values is large (and thus has many outliers), we decided to compare using the median. That is, our refined question is:

```
Do ads with greater than or equal to the median num
ber of impressions cost more?
```

Null Hypothesis: Ads that have more than the median number of impressions don't cost more.

Alternate Hypothesis: Ads that have more than the median number of impressions cost more.

We have decided to use the difference of means as a test statistic. The reason why we have decided on this test statistic specifically is because we are testing two populations: the spread of impressions on 'popular' ads, and 'unpopular' ads. We also used a permutation test to answer our question. As with the permutation test for missingness, we set the significance level to be 0.1.

After a permutation test with `N=500` repetitions, we found our p-value to be 0.014, which is below the significance level. As such, we can reject

Code

In [1]:

```
import matplotlib.pyplot as plt
import numpy as np
import os
import pandas as pd
import seaborn as sns
%matplotlib inline
%config InlineBackend.figure_format = 'retina'  # Higher r
esolution figures
```

Cleaning and EDA

Cleaning

In [110]:

```
ads_2018 = pd.read_csv('PoliticalAds_2018.csv')
ads_2019 = pd.read_csv('PoliticalAds_2019.csv')

# concatenating 2018 and 2019 data
ads = pd.concat([ads_2018, ads_2019], ignore_index=True)
```

In [123]:

```
# after checking using value_counts, all dates are in the
 same time zone UTC so we removed the 'Z'
ads['StartDate'] = ads['StartDate'].str[:-1]
ads['EndDate'] = ads['EndDate'].str[:-1]

# convert them to datetime format
ads['StartDate'] = pd.to_datetime(ads['StartDate'])
ads['EndDate'] = pd.to_datetime(ads['EndDate'])
```

Currency Conversion

Besides the datetime formatting for `StartDate` and `EndDate`, we found that the only column we needed to standardize was `Spend`, since the currencies weren't all in USD.

In [177]:

```
#viewing the currencies that show in the dataset
ads['Currency Code'].value_counts()
```

Out[177]:

```
USD    2532
EUR     708
GBP     588
CAD     400
AUD      40
Name: Currency Code, dtype: int64
```

Below are the exchange rates. We took the average of 2019.

- USD: 1
- Euro: 0.893
- GBP: 0.784
- CAD: 1.327
- AUD: 1.439

Source: ([https://www.irs.gov/individuals/international-taxpayers/yearly-average-currency-exchange-rates (https://www.irs.gov/individuals/international-taxpayers/yearly-average-currency-exchange-rates)](https://www.irs.gov/individuals/international-taxpayers/yearly-average-currency-exchange-rates)):

We divide the foreign currency by these multipliers.

In [181]:

```
# used exchange rate on 14 May, 2020
exchange_rate = {'USD': 1, 'EUR': 0.893, 'GBP': 0.784, 'CA
D': 1.327, 'AUD': 1.439}

# convert other currencies to USD
ads['Spend (USD)'] = ads['Spend'] / ads['Currency Code'].m
ap(exchange_rate)
```

EDA

Univariate Analysis

Exploring the Impressions Column

In [183]:

```
# looking at summary statistics
impressions = ads['Impressions']
impressions.describe()
```

Out[183]:

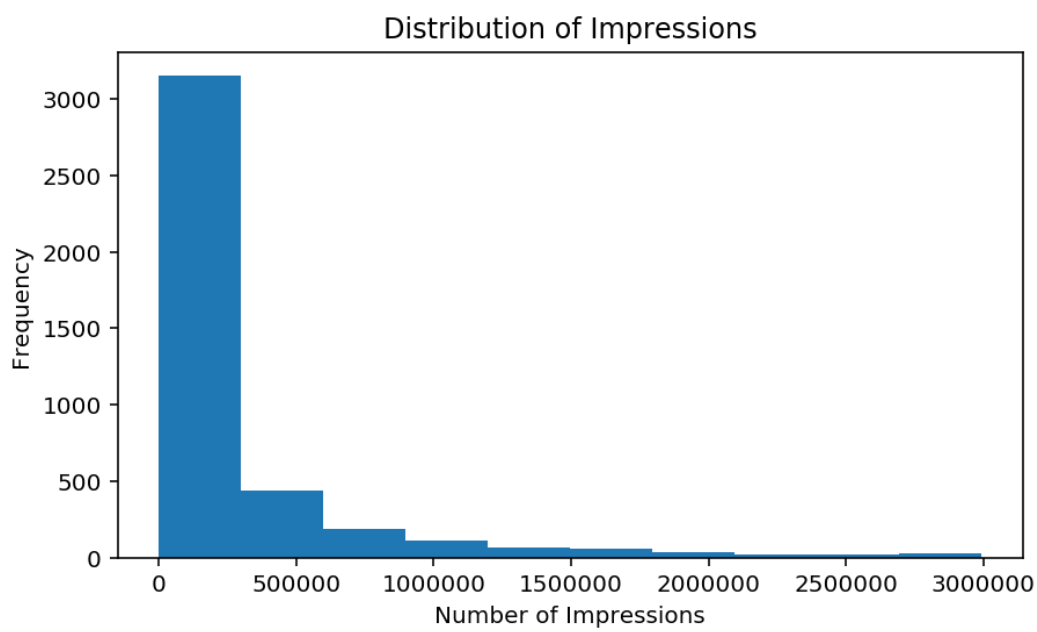
```
count    4.268000e+03
mean     7.022956e+05
std      4.944558e+06
min      1.000000e+00
25%      1.320050e+04
50%      7.268950e+04
75%      3.237462e+05
max      2.349018e+08
Name: Impressions, dtype: float64
```

In [185]:

```
# distribution of impressions (without outliers for visual
ization purposes)
impressions_vis = impressions[impressions < 3000000] #96%
 of data
impressions_vis.plot(kind = 'hist', title = 'Distribution
 of Impressions', figsize=(7, 4))
plt.xlabel('Number of Impressions')
```

Out[185]:

```
Text(0.5, 0, 'Number of Impressions')
```

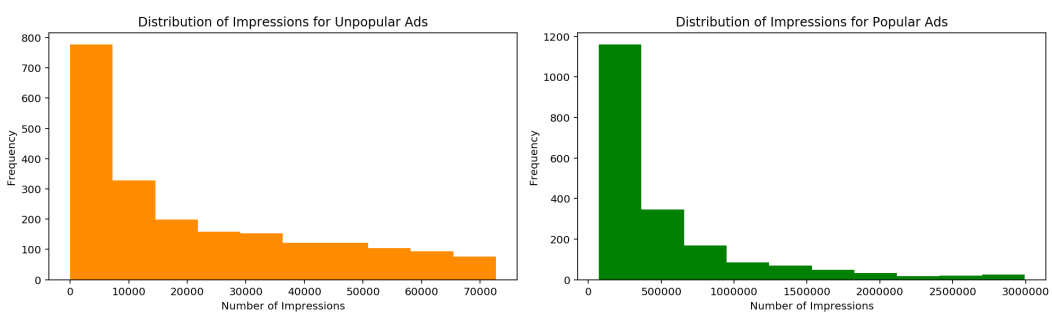


In [186]:

```
# distribution of popular vs. unpopular (without outliers
 for visualization purposes)
popular_vis = popular[popular < 3000000] #92.5% of data

fig, axes = plt.subplots(1, 2, figsize=(14,4))
ax1 = unpopular.plot(ax=axes[0], title='Distribution of Im
pressions for Unpopular Ads', kind = 'hist', color = 'dark
orange')
ax2 = popular_vis.plot(ax=axes[1], title='Distribution of
 Impressions for Popular Ads', kind = 'hist', color = 'gre
en')
plt.tight_layout()
ax1.set_xlabel('Number of Impressions')
ax2.set_xlabel('Number of Impressions')
```

Out[186]:

```
Text(0.5, 15.0, 'Number of Impressions')
```



Exploring the Spend Column

In [188]:

```
# looking at summary statistics
spend = ads['Spend (USD)']
spend.describe()
```

Out[188]:

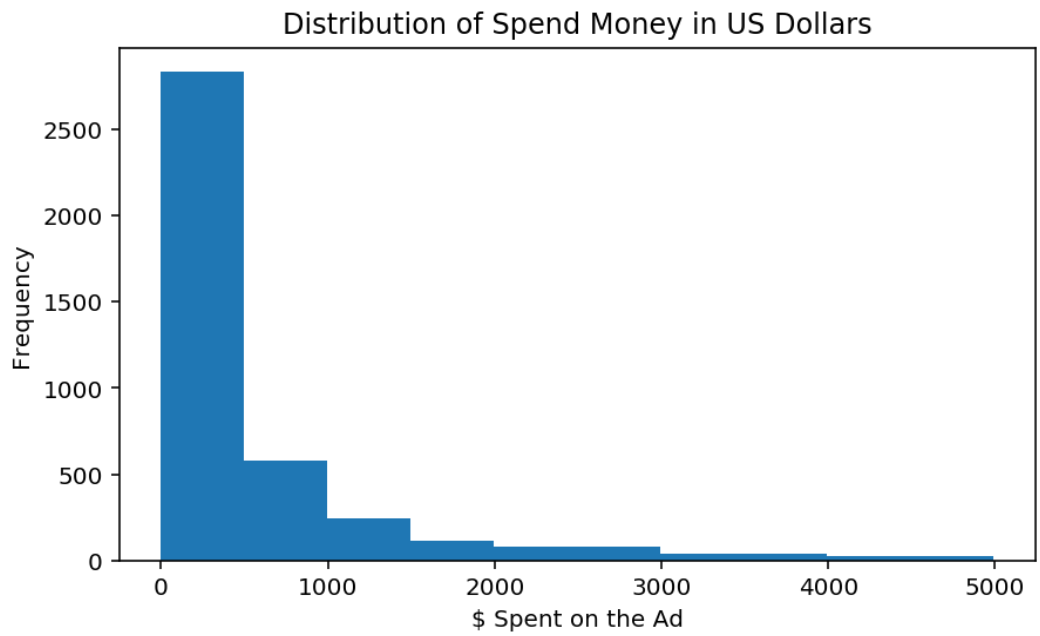
```
count      4268.000000
mean       1652.434693
std       12124.311906
min           0.000000
25%          41.335154
50%         194.924412
75%         776.000000
max      451244.000000
Name: Spend (USD), dtype: float64
```

In [189]:

```
# distribution (without outliers for visualization purpose
s)
spend_vis = spend[spend < 5000] #95% of data
spend_vis.plot(kind = 'hist', title = 'Distribution of Spe
nd Money in US Dollars', figsize=(7, 4))
plt.xlabel('$ Spent on the Ad')
```

Out[189]:

```
Text(0.5, 0, '$ Spent on the Ad')
```



Bivariate Analysis

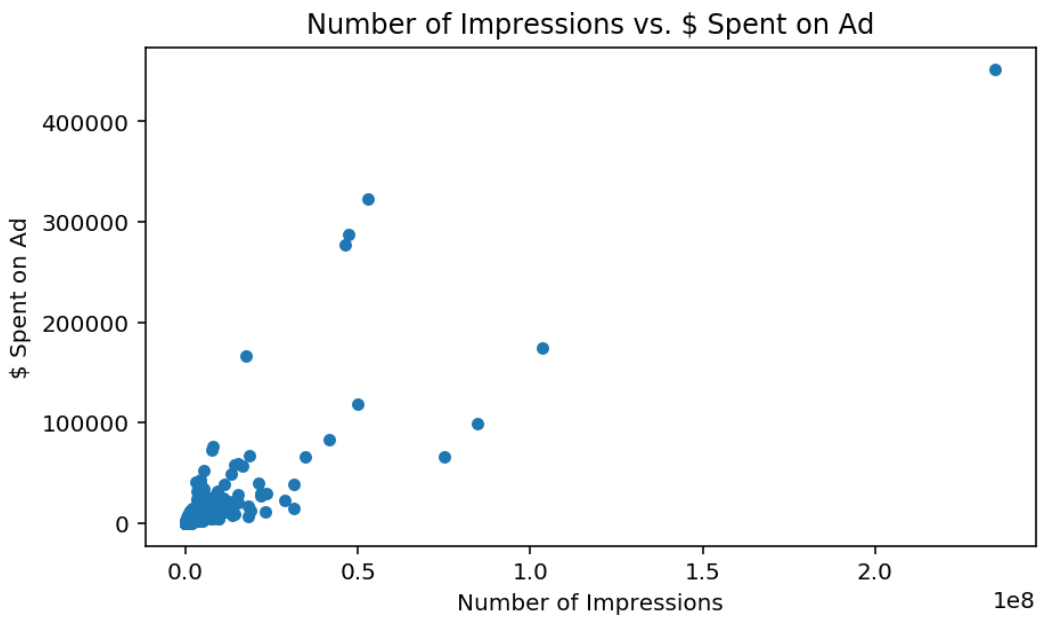
Impressions and Spend

In [191]:

```
ads.plot(kind = 'scatter', x = 'Impressions', y = 'Spend
 (USD)', title = 'Number of Impressions vs. $ Spent on Ad'
, figsize = (7, 4))
plt.ylabel('$ Spent on Ad')
plt.xlabel('Number of Impressions')
```

Out[191]:

```
Text(0.5, 0, 'Number of Impressions')
```



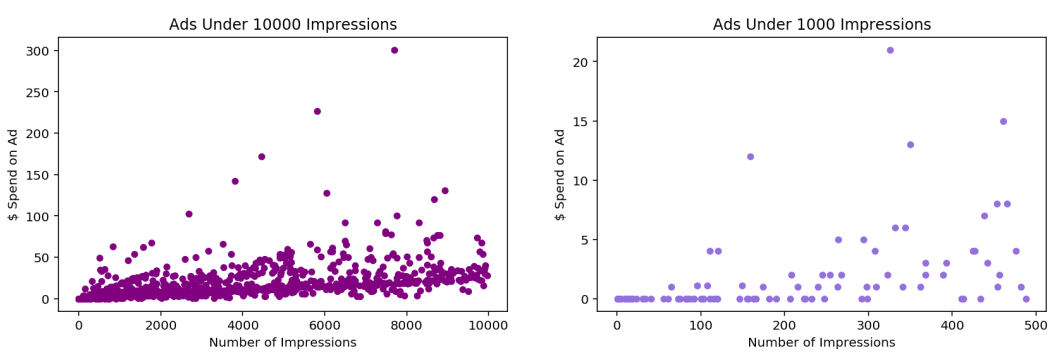
We wish to get a closer view of the cluster near the origin. Below we plot for lower numbers of impressions.

In [193]:

```
# closer look
ads2 = ads[ads['Impressions'] < 10000]
ads3 = ads[ads['Impressions'] < 500]

# plotting
fig, axes = plt.subplots(1, 2, figsize=(14,4))
ax1 = ads2.plot(ax = axes[0], kind = 'scatter', y = 'Spend
(USD)', x = 'Impressions', title = 'Ads Under 10000 Impres
sions', color = 'purple')
ax1.set_ylabel('$ Spend on Ad')
ax1.set_xlabel('Number of Impressions')
ax2 = ads3.plot(ax = axes[1], kind = 'scatter', y = 'Spend
(USD)', x = 'Impressions', title = 'Ads Under 1000 Impress
ions', color = 'mediumpurple')
ax2.set_ylabel('$ Spend on Ad')
ax2.set_xlabel('Number of Impressions')
```

Out[193]:

```
Text(0.5, 0, 'Number of Impressions')
```



Interestingly, it is quite possible to spend very little on an ad and still have a similar reach as an ad that spent much more. There is a sizeable number of ads that spend less than 50 USD and yet have more impressions than the ads that spend upwards of 150 USD.

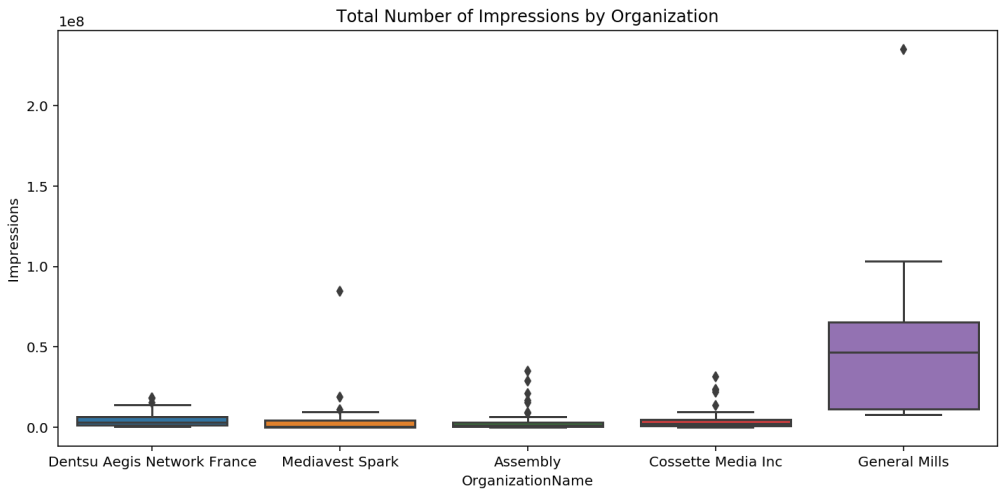
Impressions and Organizations

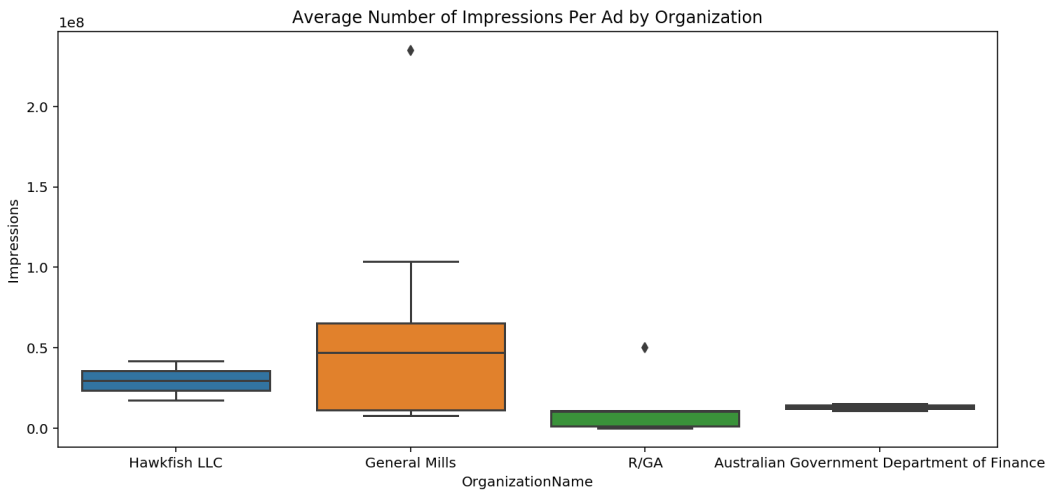
In [194]:

```
# filter for top 5 orgs who got the most impressions overa
ll
top_orgs = ads.groupby('OrganizationName').sum().sort_valu
es(by = 'Impressions', ascending = False).head(5).index
top_only = ads[ads['OrganizationName'].isin(top_orgs)]

# most impressions per ad
top_orgs_per = ads.groupby('OrganizationName').mean().sort
_values(by = 'Impressions', ascending = False).head(4).ind
ex
top_only_per = ads[ads['OrganizationName'].isin(top_orgs_p
er)]

# plotting
fig, axes = plt.subplots(2, 1, figsize = (12, 12))
ax1 = sns.boxplot(ax = axes[0], data = top_only, x = 'Orga
nizationName', y = 'Impressions')
ax1.set_title('Total Number of Impressions by Organizatio
n')
ax2 = sns.boxplot(ax = axes[1], data = top_only_per, x =
'OrganizationName', y = 'Impressions')
ax2.set_title('Average Number of Impressions Per Ad by Org
anization')
```

Out[194]:

```
Text(0.5, 1.0, 'Average Number of Impressions
Per Ad by Organization')
```





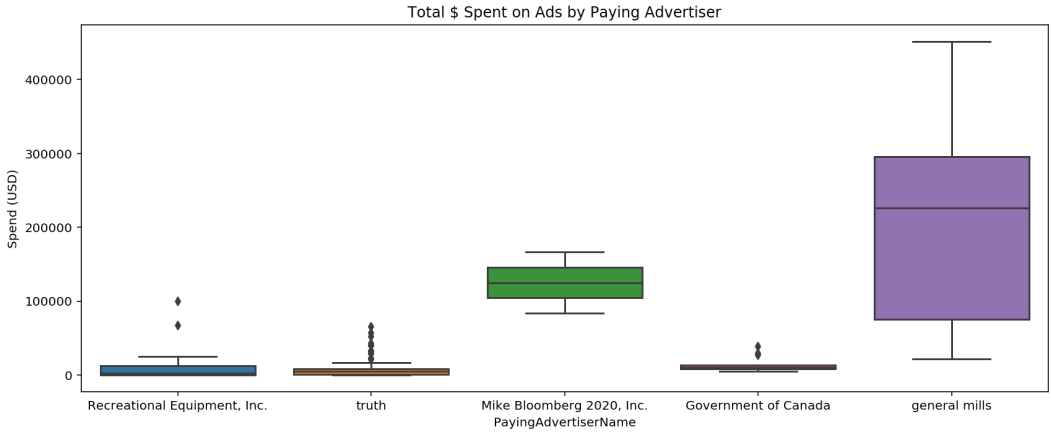
Spend and Paying Advertiser

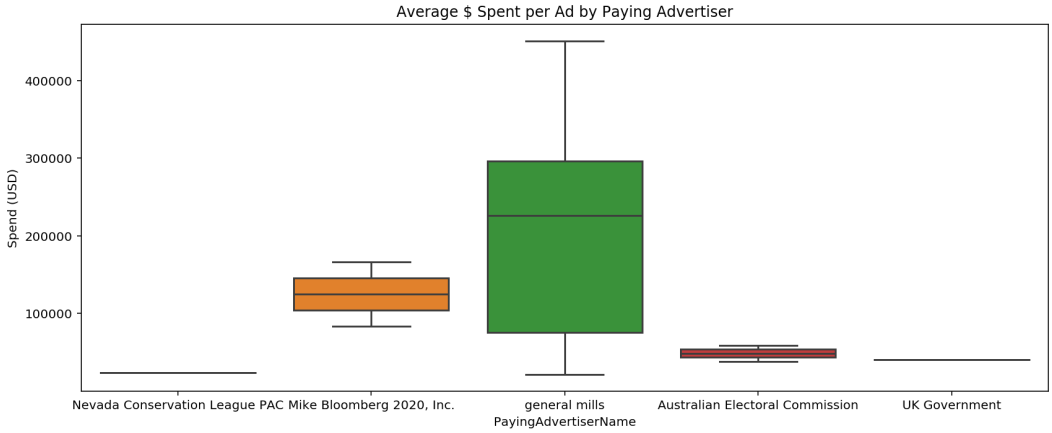
In [196]:

```
# filter for top 5 advertisers who paid the most money
top_adv = ads.groupby('PayingAdvertiserName').sum().sort_v
alues(by = 'Spend (USD)', ascending = False).head(5).index
top_adv_only = ads[ads['PayingAdvertiserName'].isin(top_ad
v)]

# most money per ad
top_adv_per = ads.groupby('PayingAdvertiserName').mean().s
ort_values(by = 'Spend (USD)', ascending = False).head(5).
index
top_adv_only_per = ads[ads['PayingAdvertiserName'].isin(to
p_adv_per)]

# plotting
fig, axes = plt.subplots(2, 1, figsize = (14, 12))
ax1 = sns.boxplot(ax=axes[0], data = top_adv_only, x = 'Pa
yingAdvertiserName', y = 'Spend (USD)')
ax1.set_title('Total $ Spent on Ads by Paying Advertiser')
ax2 = sns.boxplot(ax=axes[1], data = top_adv_only_per, x =
'PayingAdvertiserName', y = 'Spend (USD)')
ax2.set_title('Average $ Spent per Ad by Paying Advertise
r')
```

Out[196]:

```
Text(0.5, 1.0, 'Average $ Spent per Ad by Pay
ing Advertiser')
```





Aggregating

In [198]:

```
# making pivot table of averages
averages = pd.pivot_table(ads, index = ['PayingAdvertiserN
ame', 'OrganizationName'], values = ['Impressions', 'Spend
(USD)'],
              aggfunc = {'Impressions': np.mean, 'Spend (U
SD)': np.mean})

# making pivot table of sums
sums = pd.pivot_table(ads, index = ['PayingAdvertiserName'
, 'OrganizationName'], values = ['Impressions', 'Spend (US
D)'],
              aggfunc = {'Impressions': np.sum, 'Spend (US
D)': np.sum})

# averages sorted by largest average impressions and spend
averages.sort_values(by = ['Impressions', 'Spend (USD)'],a
scending = False).head()
```

Out[198]:

PayingAdvertiserName	OrganizationName	Impressions
general mills	General Mills	6.411373e+07
Mike Bloomberg 2020, Inc.	Hawkfish LLC	2.958974e+07
Planned Parenthood	R/GA	1.459777e+07
Ministère de l'Intérieur	Dentsu Aegis Network France	1.342636e+07
Australian Electoral Commission	Australian Government Department of Finance	1.320552e+07

In [200]:

```
# sorted by largest total impressions and spend
sums.sort_values(by = ['Impressions', 'Spend (USD)'],ascen
ding = False).head()
```

Out[200]:

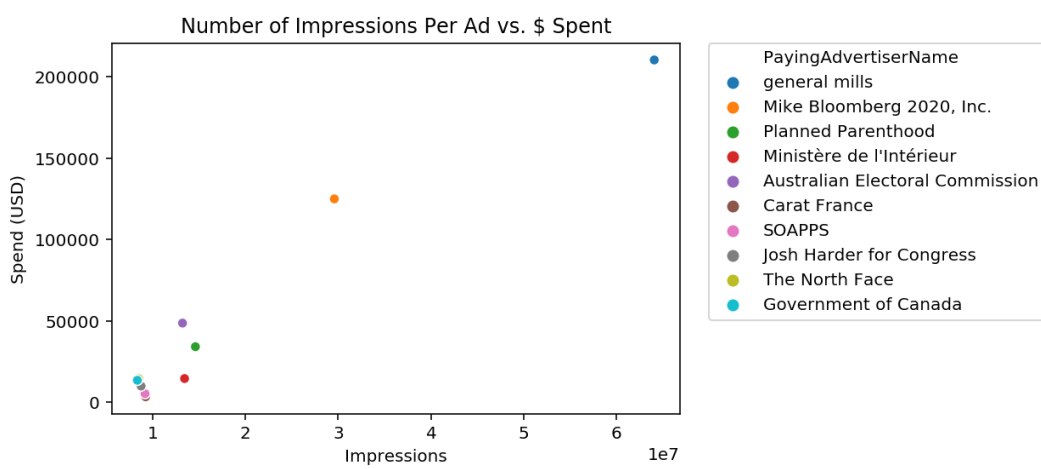
PayingAdvertiserName	OrganizationName	Impressions	
general mills	General Mills	512909841	1
truth	Assembly	268204613	8
Recreational Equipment, Inc.	Mediavest Spark	151681954	2
SIG	Dentsu Aegis Network France	144208664	1
Government of Canada	Cossette Media Inc	133347331	2

In [202]:

```
# plotting impressions vs. spend by advertiser for top pay
ing advertisers
top = averages.sort_values(by = ['Impressions', 'Spend (US
D)'],ascending = False).head(10)
top.reset_index(inplace = True)


sns.scatterplot(data=top, x='Impressions', y='Spend (USD)'
, hue='PayingAdvertiserName')
plt.legend(bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=
0.)
plt.title('Number of Impressions Per Ad vs. $ Spent')
```

Out[202]:

```
Text(0.5, 1.0, 'Number of Impressions Per Ad
vs. $ Spent')
```



Assessment of Missingness

Segments

We chose to test `Segments` against `Spend` and `Impressions`, since we decided those columns were relevant—for example, the greater reach an ad has or the more an organization spends, the more likely it will not leave `Segments` out.

In [227]:

```
# taking only relevant columns
segments = ads.loc[:,['Spend (USD)', 'Impressions', 'Segme
nts']]
segments['seg_null'] = segments['Segments'].isnull()
```

In [228]:

```
# our test statistic is the difference in means
def diff_in_means(df, col):
    grouped = df.groupby('seg_null').mean()
    return grouped[col].diff().abs().iloc[-1]
```

An initial glance shows that, on average, an ad with a NaN value in `Segments` has more impressions and more money spent on it.

In [229]:

```
segments.groupby('seg_null').mean()
```

Out[229]:

seg_null	Spend (USD)	Impressions
False	1763.951152	735541.098507
True	1384.100996	622299.529130

We conducted a permutation test on `Segments` to see if it was dependent on `Spend` or `Impressions`, with a significance level of 0.05.

In [230]:

```
N = 200
columns = ['Spend (USD)', 'Impressions']

for col in columns:
    obs = diff_in_means(segments, col)
    diff_of_means = []

    for i in range(N):
        shuffled_col = segments[col].sample(replace=False,
frac=1).reset_index(drop=True)
        shuffled = segments.assign(**{'Shuffled': shuffled
_col})

        current = diff_in_means(shuffled, 'Shuffled')
        diff_of_means.append(current)

    print(col + ': ' + str(np.mean(diff_of_means >= obs)))
```

```
Spend (USD): 0.4
Impressions: 0.58
```

Neither p-value is less than our significance level (0.1), thus we conclude that the correlation between `Segments` and `Spend` / `Impressions` is **not statistically significant**. This would mean that the missingness mechanism for `Segments` is NMAR, since it is not dependent on any observable data in the dataset.

Candidate Ballot Information

We chose to test `CandidateBallotInformation` against `Spend` and `Impressions` too. We will be using the same test statistic, difference_in_means, and the same significance level 0.1.

In [218]:

```
# taking only relevant columns
candidate = ads.loc[:,['Spend (USD)', 'Impressions', 'Cand
idateBallotInformation']]
candidate['is_null'] = candidate['CandidateBallotInformati
on'].isnull()
```

In [219]:

```
# our test statistic is the difference in means
def diff_in_means(df, col):
    grouped = df.groupby('is_null').mean()
    return grouped[col].diff().abs().iloc[-1]
```

Again, another cursory look at the dataframe.

Interestingly, if we compare this groupby table to the one above (with `Segments`), the difference of the mean of `Impressions` is much higher.

In [220]:

```
candidate.groupby('is_null').mean()
```

Out[220]:

is_null	Spend (USD)	Impressions
False	1145.295209	357292.008772
True	1730.603840	755473.574094

In [226]:

```
# calculate the p-values for spend and impressions against
candidate ballot information

N = 200
columns = ['Spend (USD)', 'Impressions']

for col in columns:
    obs = diff_in_means(candidate, col)
    diff_of_means = []

    for i in range(N):
        shuffled_col = candidate[col].sample(replace=False
, frac=1).reset_index(drop=True)
        shuffled = candidate.assign(**{'Shuffled': shuffle
d_col})

        current = diff_in_means(shuffled, 'Shuffled')
        diff_of_means.append(current)

    print(col + ': ' + str(np.mean(diff_of_means >= obs)))
```

```
Spend (USD): 0.305
Impressions: 0.075
```

While the p-value of `Spend` is 0.305 and thus above the significance level, the p-value for comparing `Impressions` is 0.075, which less than 0.1. This means that the correlation between `Spend` and `CandidateBallotInformation` is **not statistically significant**, while the correlation between `Impressions` and `CandidateBallotInformation` **is statistically significant**.

As such, we concluded that `CandidateBallotInformation` is MAR dependent on the column `Impressions` .

Hypothesis Test

As a reminder, this is our question:

```
Do ads with greater than or equal to the median num
ber of impressions (around 72,000) cost more?
```

Null Hypothesis: Ads that have more than the median number of impressions don't cost more.

Alternate Hypothesis: Ads that have more than the median number of impressions cost more.

- Test statistic: difference of means
- Significance level: 0.1

We will use a permutation test.

In [206]:

```
# test statistic is difference in means
def diff_in_means(df, col):
    grouped = df.groupby('Popular').mean()
    return grouped[col].diff().abs().iloc[-1]
```

In [209]:

```
# drop all unnecessary columns
ht = ads.loc[:,['Spend (USD)', 'Impressions']]
```

In [210]:

```
# check if an ad has more impressions than the median (or
 equal to)
median = ht['Impressions'].median()
ht['Popular'] = ht['Impressions'] >= median
```

In [211]:

```
# permutation test
N = 500
obs_stat = diff_in_means(ht, 'Spend (USD)')
diff_means = []

for i in range(N):
    shuffled_col = ht['Impressions'].sample(replace=False,
frac=1).reset_index(drop=True)
    shuffled = ht.assign(**{'Shuffled': shuffled_col})

    current = diff_in_means(shuffled, 'Shuffled')
    diff_means.append(current)

print('P-Value: ' + str(np.mean(diff_means <= obs_stat)))
```

```
P-Value: 0.014
```

Since our p-value is less than the significance level, we can conclude that the correlation between the number of impressions on an ad and its cost is statistically significant. That is, we can reject our null hypothesis and find that there is a link between number of impressions and cost.

Prediction Problem

Modeling Introduction

Our prediction problem will be to predict the reach of an advertisement, which is a regression problem.

We can define 'reach' as the number of views garnered by an individual advertisement. According to the readme file for ads, `Impressions` is the "Number of times the Ad has been viewed by Snapchatters." As such, `Impressions` would be the ideal target variable of choice.

Since this is a regression problem, our objective is to maximize the accuracy of our models, which we will measure by finding their R^2 and RMSE values. These metrics will tell us how good our fit is and the extent of our errors.

Baseline Model

For our baseline model, we used 4 features to predict impressions: Spend (quantitative), Start Month (ordinal), Start Year (ordinal), and Currency Code (nominal).

We chose these features based on our exploratory analysis from the previous project in which we found that money spent has some correlation with impressions. The currency used to pay for the ad could also narrow down the country/politics that the ad is addressing, and since some politics are more relevant than others (e.g. USA politics are more well-known globally compared to other countries), certain political ads could garner more impressions. Additionally, we decided to include

features of time (month and year) to take into account potential political events during certain times that may boost or decrease political interest and thus impressions.

Our single quantitative column, the `Spend` column was adjusted to US dollars for consistency and our categorical columns were one-hot encoded. As it is the most straightforward approach, we used a linear regression model to predict the number of impressions.

After training and fitting this baseline model, it scored roughly:

R^2: -0.0017220599358778177

RMSE: 4303282.355449376

These are a very low (negative even!) R^2 and a very high RMSE, the former indicating that our baseline model is a poor fit for the data, and the latter indicating that our model has high error. It is clear that these R^2 and RMSE values are not the best scores and that our model needed to be improved.

Final Model

We added several more engineered features in addition to what we had in our baseline model:

- We added `OrganizationName` because we saw from previous EDA in project 3 that the number of impressions is potentially correlated with certain organizations. A large organization typically addresses larger, more well known issues. They also have many political advertisements, and said advertisements may have more of a reach compared to the advertisements of smaller organizations. However, there are many organizations who only post a political ad once or twice, which makes predicting those rather unreliable. To

address that issue, we have categorized any organization that has less than 2% of the total ads listed in the dataset as `Other`. After that, we one-hot encoded this transformed column and added it to our final model.
- We performed PCA on all the categorical columns we previously used in the baseline model--this serves to remove any unnecessary information, which in turn reduces noise and improves accuracy of our model.
- We used some string manipulation to take the country/state of the organization from `BillingAddress`. After some deliberation, we thought that while `Currency Code` might have given us a good idea of where an organization or ad may have come from (which in turn could help us predict if the politics of said place were significant enough to result in a high number of impressions), looking at the billing address of these organizations would be more accurate than the currency they use. This transformed column was thus one-hot encoded and included in the final model.
- The `Spend (US)` column was z-scaled to ensure a consistent scale across the data, especially because the Spend column is scaled so differently from our other features.

We decided to test LinearRegression against KNeighborsRegressor to see which was a better fit for our data. For the LinearRegression model, we left the parameters at the default bools because they were either irrelevant (copy_X and n_jobs) or unnecessary (fit_intercept and normalize). For the KNeighborsRegressor, we determined that `n_neighbors` and `weights` were the most relevant parameters, and tested said parameters using GridSearchCV. We found that the ideal parameters were `n_neighbors=9`, `weights='distance'`.

After training and fitting both models, we got the following scores:

Linear Regression Model

- **R^2:** 0.3058731077700818
- **RMSE:** 3582165.600469051

KNearest Neighbors (final) Model

- **R^2:** 0.6374423618429232
- **RMSE:** 2588896.887386254

Clearly, a KNearest Neighbors model was the best model for this problem, even if it wasn't the simplest--there is a significant difference in R^2 and RMSE values between the Linear Regression and KNearest Neighbors models. As such, we chose to use the KNearest Neighbors model as the final model. The features we decided to engineer also clearly improved the accuracy of our final model, as there is a significant improvement in R^2 and RMSE values compared to the baseline model. The R^2 value is higher compared to the baseline model, which indicates that this model is a better fit for the dataset, and the RMSE value is lower, which indicates that this model has less error compared to the baseline model.

Fairness Evaluation

We chose to evaluate our final model for fairness against `Gender` --does it perform better against female-targeted ads or male-targeted ads? Since this is a regression model, R^2 was the most obvious choice as a parity measure, as it is a good measure of accuracy for linear regression models and RMSE is a more difficult metric to interpret and compare.

Null Hypothesis: The R^2 value of the model for both subsets of ads is the same. As such, our model is fair.

Alternate Hypothesis: The R^2 value of the model for the male and female subsets of ads is different. This means our model is unfair, and biased to a certain subset.

We determined our test statistic to be the difference in R^2 values between the model's scores on the female dataset versus the male dataset:

```
r_diff = r_squared_m - r_squared_f
```

which we calculated to be 0.14924058857631828.

Since the NaN values in this column mean that the advertisement does not have a specific target gender, we decided to group them into both subsets. That is, all NaN values in the column were replaced with `ALL`, and the female subset would contain ads targeted to `ALL` and `FEMALE` audiences, while the male subset would contain ads targeted to `ALL` and `MALE` audiences.

After creating our subsets, we set our significance level to be `0.05` and for our permutation test to occur `N=100` times. We got a **p-value of `0.5530063126824178`**, which is so far from the significance level that we undeniably fail to reject the null hypothesis. As such, we cannot say that the final model performs better on a certain subset (though we can't definitively say that the final model is absolutely fair to all demographics).

Code

In [189]:

```
import matplotlib.pyplot as plt
import numpy as np
import os
import pandas as pd
import seaborn as sns
%matplotlib inline
%config InlineBackend.figure_format = 'retina'  # Higher r
esolution figures

from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import FunctionTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.impute import SimpleImputer
from statistics import mean
```

In [2]:

```
import warnings
warnings.filterwarnings('ignore')
```

Cleaning

In [3]:

```
ads_2018 = pd.read_csv('PoliticalAds_2018.csv')
ads_2019 = pd.read_csv('PoliticalAds_2019.csv')

# concatenating 2018 and 2019 data
ads = pd.concat([ads_2018, ads_2019], ignore_index=True)

# get start month and year
dates = pd.to_datetime(ads['StartDate'])
ads['StartMonth'] = dates.apply(lambda x: x.month)
ads['StartYear'] = dates.apply(lambda x: x.year)

# used exchange rate on 14 May, 2020
exchange_rate = {'USD': 1, 'EUR': 0.893, 'GBP': 0.784, 'CA
D': 1.327, 'AUD': 1.439}

# convert other currencies to USD
ads['Spend (USD)'] = ads['Spend'] / ads['Currency Code'].m
ap(exchange_rate)
```

Generating Test and Train Data

In [359]:

```
cols = ['Spend (USD)', 'Currency Code', 'StartMonth', 'Sta
rtYear', 'BillingAddress', 'OrganizationName']
X = ads[cols]
y = ads.Impressions

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.25)
```

Baseline Model

In [368]:

```
# one-hot categorical cols
cat_feat = ['Currency Code']

cat_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder())
])
```

In [373]:

```
# column transformer and pipeline for baseline model
preproc = ColumnTransformer(remainder = 'drop',
                            transformers=[('nothing', Func
tionTransformer(), ['Spend (USD)', 'StartMonth', 'StartYea
r']),
                                          ('cat', cat_tran
sformer, cat_feat)])

pl = Pipeline(steps=[('preprocessor', preproc), ('regresso
r', LinearRegression())])
```

In [374]:

```
# fitting and results
pl.fit(X_train, y_train)

# calculate accuracy of baseline model
r_squared = pl.score(X_test, y_test)
preds = pl.predict(X_test)
rmse = np.sqrt(np.mean((preds - y_test)**2))

print ("R^2: %s" % r_squared)
print ("RMSE: %s" % rmse)
```

```
R^2: -0.0017220599358778177
RMSE: 4303282.355449376
```

Final Model

In [360]:

```
# feature 1: categorizes org into "other" if less than 2%
 (and perform pca on it)
def check_org(df):
    orgs = df['OrganizationName'].apply(lambda x: 'Other'
if ((ads['OrganizationName'] == x).sum() / len(ads)) < 0.1
else x)
    return pd.DataFrame(orgs)

orgs = Pipeline(steps=[('check-org', FunctionTransformer(c
heck_org, validate=False)),
                       ('one-hot', OneHotEncoder(sparse=Fa
lse)),
                       ('pca', PCA(svd_solver='full', n_co
mponents=0.95))])

final_orgs = ColumnTransformer(transformers=[('check', org
s, ['OrganizationName'])])
```

In [361]:

```
# feature 2: pca on one-hot
cat_feat = ['Currency Code', 'StartMonth', 'StartYear']

cat_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder(sparse=False)),
    ('pca', PCA(svd_solver='full', n_components=0.95))
])
```

In [362]:

```
# feature 3: add the country/state from the billing addres
s (and perform pca on it)
def take_address(df):
    return pd.DataFrame(df['BillingAddress'].str.findall(r
"[A-Za-z]{2}$").explode())

pl_address = Pipeline([('pull-address', FunctionTransforme
r(take_address, validate=False)),
                       ('one-hot-address', OneHotEncoder(s
parse=False, handle_unknown='ignore')),
                       ('pca', PCA(svd_solver='full', n_co
mponents=0.95))])
```

In [363]:

```
# feature 4: z-scale `Spend (USD)` col
num_feat = ['Spend (USD)']
num_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())   # z-scale
])
```

In [364]:

```
# column transformer for final model
preprocfinal = ColumnTransformer(remainder='passthrough',
                                 transformers=[
                                     ('num', num_transfor
mer, num_feat),
                                     ('bill-address', pl_
address, ['BillingAddress']),
                                     ('freq-org', final_o
rgs, ['OrganizationName']),
                                     ('pca-onehot', cat_t
ransformer, cat_feat)])
```

In [365]:

```
# linear regression model: fitting and scoring
pl_linreg = Pipeline(steps=[('preproc', preproc_final), (
'lin-reg', LinearRegression())])

# fitting
pl_linreg.fit(X_train, y_train)

# calculate accuracy of linear regression model
r_squared = pl_linreg.score(X_test, y_test)
preds = pl_linreg.predict(X_test)
rmse = np.sqrt(np.mean((preds - y_test)**2))

print ("Scores for LinearRegression model")
print ("R^2: %s" % r_squared)
print ("RMSE: %s" % rmse)
```

```
Scores for LinearRegression model
R^2: 0.3058731077700818
RMSE: 3582165.600469051
```

In [366]:

```
# finding ideal parameters for KNearestNeighbors model
param_data = preproc_final.fit_transform(X_train, y_train)

parameters = {
    'n_neighbors': [3, 5, 7, 9, 11],
    'weights':['uniform', 'distance']
}

clf = GridSearchCV(KNeighborsRegressor(), parameters, cv=5
, verbose=1, n_jobs=-1)
clf.fit(param_data, y_train)
clf.best_params_
```

```
Fitting 5 folds for each of 10 candidates, to
talling 50 fits

[Parallel(n_jobs=-1)]: Using backend LokyBack
end with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done  50 out of  50 |
elapsed:    0.3s finished
```

Out[366]:

```
{'n_neighbors': 9, 'weights': 'distance'}
```

In [367]:

```
# k nearest neighbors regression model: fitting and scorin
g
pl_kne = Pipeline(steps=[('preproc', preproc_final),
                         ('k-reg', KNeighborsRegressor(n_n
eighbors=9, weights= 'distance'))])

# fitting
pl_kne.fit(X_train, y_train)

# calculate accuracy of k nearest neighbors model
r_squared = pl_kne.score(X_test, y_test)
preds = pl_kne.predict(X_test)
rmse = np.sqrt(np.mean((preds - y_test)**2))

print ("Scores for KNeighborsRegressor model")
print ("R^2: %s" % r_squared)
print ("RMSE: %s" % rmse)
```

```
Scores for KNeighborsRegressor model
R^2: 0.6374423618429232
RMSE: 2588896.887386254
```

Fairness Evaluation

In [375]:

```
# binarizing gender
ads['Gender'] = ads['Gender'].replace({np.nan: 'ALL'})

female = ads[(ads['Gender'] == 'ALL') | (ads['Gender'] ==
'FEMALE')]
male = ads[(ads['Gender'] == 'ALL') | (ads['Gender'] == 'M
ALE')]
```

In [376]:

```
# function to calculate r^2 value of a dataset
def calc_r_squared(df):
    cols = ['Spend (USD)', 'Currency Code', 'StartMonth',
'StartYear', 'OrganizationName', 'BillingAddress']
    X = df[cols]
    y = df.Impressions

    X_train, X_test, y_train, y_test = train_test_split(X,
y, test_size=0.25)

    # fitting and results
    pl_kne.fit(X_train, y_train)

    # calculate accuracy
    r_squared = pl_kne.score(X_test, y_test)

    return r_squared
```

In [377]:

```
# calculating test statistic, which is difference in R^2 v
alues among the two subsets
r_squared_m = calc_r_squared(male)
r_squared_f = calc_r_squared(female)
r_diff = r_squared_m - r_squared_f

print ("Male R^2: %s" % r_squared_m)
print ("Female R^2: %s" % r_squared_f)
print ("Test Statistic: %s" % r_diff)
```

```
Male R^2: 0.5530063126824178
Female R^2: 0.40376572410609957
Test Statistic: 0.14924058857631828
```

In [378]:

```
# permutation test, using R^2 to determine accuracy
N = 100
r_diffs = []

for i in range(N):
    diff = calc_r_squared(male) - calc_r_squared(female)

    r_diffs.append(diff)
```

In [379]:

```
# finding p-value, which resulted in 0.70--much greater th
an our set significance level
p_value = np.mean(r_diffs <= r_diff)
print ("P-Value: %s" % r_squared_m)
```

```
P-Value: 0.5530063126824178
```

In [ ]:

```

```